DBT Day 4

Shivam Zanjurne · Mon, 12 Oct 26 · Relational algebra, set operations, all join types (incl. self + 3-table), window functions, LAG/LEAD, CTEs (simple, chained, recursive)

Table of Contents

1. Relational Algebra Basics
2. Set Operations: UNION, INTERSECT, EXCEPT
3. What Is a Join?
4. Setup: Department and Student Tables
5. CROSS JOIN
6. INNER JOIN and Equi Join
7. Old Join Syntax (WHERE) and USING
8. NATURAL JOIN
9. Non-Equi Join
10. LEFT JOIN and RIGHT JOIN
11. FULL OUTER JOIN in MySQL
12. SELF JOIN
13. Joining Three Tables
14. Window Functions vs GROUP BY
15. OVER, PARTITION BY, ORDER BY
16. Running Totals and Moving Windows
17. ROW_NUMBER, RANK, DENSE_RANK, NTILE
18. Window Aggregates: SUM, AVG, COUNT
19. LAG and LEAD
20. CTEs — Common Table Expressions
21. Day Wrap and Practice Checklist

Relational Algebra Basics

What is relational algebra?

Relational algebra is a **procedural** query language for relational databases. Procedural means you describe *how* to get the result step by step (select these rows, then project these columns, then combine sets…), as opposed to pure SQL which is more *declarative* ("give me this result").

Why learn it?

- Interview / exam theory often asks for the **symbol** and the **SQL equivalent**
- It makes joins and set ops feel less magical — they are just algebraic operations on tables (relations)

Selection — σ (sigma)

Selection filters **rows** that satisfy a condition. Symbol: **σ** (sigma).

Relational algebra (read aloud: "sigma of city = Pune on Student"):

```
σ city = 'Pune' (Student)
```

SQL equivalent — the `WHERE` clause:

```sql
SELECT *
FROM student
WHERE city = 'Pune';
```

Idea	Meaning
σ	Keep only rows that pass the condition
SQL	`WHERE`

> Selection does **not** drop columns — it drops **rows**.

Projection — π (pi)

Projection picks **columns**. Symbol: **π** (pi).

```
π name, city (Student)
```

SQL:

```sql
SELECT student_name, city
FROM student;
```

Idea	Meaning
π	Keep only listed columns (and remove duplicate rows in pure algebra)
SQL	Column list in `SELECT`

You can combine them mentally: “project name, city after selecting city = Pune”:

```sql
SELECT student_name, city
FROM student
WHERE city = 'Pune';
-- σ first (WHERE), then π (SELECT list)
```

Set Operations: UNION, INTERSECT, EXCEPT

These operate on the **results of two SELECT queries**, not on arbitrary mismatched shapes. Rules you must follow:

1. **Same number of columns** in both SELECTs

2. **Compatible types** column-by-column
3. Matching is by **position**, not by column name — so use **aliases** when names differ (`student_name AS name`)

Class used `student` and `student_backup` with common projected columns: `id` , `name` , `course` .

UNION — all distinct rows from both sides

Symbol: ∪ (capital U shape).

Meaning: rows that appear in **query A or query B** (or both). **Duplicates across the two results appear only once.**

```
SELECT id, student_name AS name, course FROM student
UNION
SELECT id, name, course FROM student_backup;
```

- Rows only in `student` → included
- Rows only in `student_backup` → included
- Rows in **both** (same values in all selected columns) → included **once**

UNION ALL — keep duplicates

Same as UNION, but **does not remove** duplicates. If Gauri appears in both tables with the same `(id, name, course)` , you see her **twice**.

```
SELECT id, student_name AS name, course FROM student
UNION ALL
SELECT id, name, course FROM student_backup;
```

	UNION	UNION ALL
Common row in both tables	Once	Twice (once from each)
Speed	Slightly slower (dedup work)	Faster
Use when	You want a clean set	You want every contribution counted

> "Duplicate" here means **the whole selected row** matches — same id **and** same name **and** same course. Two "Ritesh" rows with different courses are **not** the same row for UNION.

INTERSECT — only rows in both

Symbol: ∩ (upside-down U).

Meaning: rows present in **both** result sets (exact match on all selected columns).

```sql
SELECT id, student_name AS name, course FROM student
INTERSECT
SELECT id, name, course FROM student_backup;
```

Class result idea: Ritesh, Gauri, Rohit, Vidya — present in both tables with the same projected values.

Order of the two SELECTs does **not** change INTERSECT's meaning (A ∩ B = B ∩ A).

EXCEPT (difference) — in first, not in second

Symbol: − (minus). SQL keyword: `EXCEPT` (MySQL 8.0.31+; some systems use `MINUS`).

Meaning: rows in the **first** SELECT that are **not** in the **second**.

```sql
-- In student, but NOT in student_backup
SELECT id, student_name AS name, course FROM student
EXCEPT
SELECT id, name, course FROM student_backup;
-- e.g. Gauri (id 1), Akshay, Aniket — unique to first table

-- Swap order → only what backup has that student doesn't
SELECT id, name, course FROM student_backup
EXCEPT
SELECT id, student_name AS name, course FROM student;
-- e.g. only Shreya
```

Order matters: `A EXCEPT B` ≠ `B EXCEPT A`.

Picture in your head

```
student              student_backup
  A B C D E            B C F

UNION:        A B C D E F     (each once)
UNION ALL:    A B C D E B C F (B,C twice)
INTERSECT:    B C
EXCEPT:       A D E           (first minus second)
```

Alias tip from class

If left query has `student_name` and right has `name`, alias so humans (and result headers) stay clear:

```sql
SELECT id, student_name AS name, course FROM student
UNION
SELECT id, name, course FROM student_backup;
```

The **result column names** come from the **first** SELECT. Values still match by position even if names differ.

What Is a Join?

> **Join** combines **rows** from two or more tables using a **related column** or a **join condition**.

You already saw the *relationship* (foreign key) on Day 3 — passport registration ↔ applications. Joins are how you **query** that relationship: "show student name **with** department name," "show every student even if no department," etc.

Relationship	Join role
Parent / child via FK	Usually join on `parent.pk = child.fk`
One-to-many	One parent row can match many child rows in the result

Which join to pick? Ask: "Do I need only matches, or also orphans from the left, or from the right, or every pairing?"

Setup: Department and Student Tables

Class built these to practice joins.

```
CREATE TABLE department (
    department_id   INT PRIMARY KEY,
    department_name VARCHAR(20) NOT NULL
);

CREATE TABLE student (
    student_id     INT PRIMARY KEY,
    name           VARCHAR(25) NOT NULL,
    department_id  INT,                    -- FK; can be NULL (not yet admitted)
    mentor_id      INT,                    -- self-FK for SELF JOIN later
    CONSTRAINT fk_student_department
        FOREIGN KEY (department_id)
        REFERENCES department(department_id),
    CONSTRAINT fk_student_mentor
        FOREIGN KEY (mentor_id)
        REFERENCES student(student_id)
);
```

Why name the FK constraints? So you can later:

```
ALTER TABLE student DROP FOREIGN KEY fk_student_department;
```

Without a name, dropping a specific FK is painful.

Sample idea (matches class narrative)

department_id	department_name
10	Computer
20	IT
30	Electronics
40	Mechanical

Students: ~12 rows; some have `department_id` 10/20/30; **Sneha / Isha** style rows may have `department_id = NULL` (no admission yet). **Nobody** in Mechanical (40) — important for RIGHT JOIN demos.

```
INSERT INTO department VALUES
    (10, 'Computer'), (20, 'IT'), (30, 'Electronics'), (40, 'Mechanical');

-- Example students (adjust to your chat-box data)
INSERT INTO student (student_id, name, department_id, mentor_id) VALUES
    (1,  'Rahul',  10, NULL),
    (2,  'Priya',  20, 1),
    (3,  'Amit',   10, 1),
    (4,  'Sneha',  NULL, 2),
    (5,  'Karan',  30, 1),
    (6,  'Neha',   20, 2),
    (7,  'Vikram', 10, 3),
    (8,  'Rohan',  20, NULL),
    (9,  'Isha',   NULL, NULL);
-- ... more rows as in class (12 total)
```

Parent / child: one department → many students. `department` is parent; `student.department_id` is child FK. PK is unique + NOT NULL; FK **may** be duplicate or NULL.

CROSS JOIN

Definition

CROSS JOIN pairs **every row** of the first table with **every row** of the second. **No join condition.**

If table A has 12 rows and B has 4 → result has **12 × 4 = 48** rows (Cartesian product).

```
SELECT s.name, d.department_name
FROM student AS s
CROSS JOIN department AS d;
```

Rahul appears **four** times (once with each department); Priya four times; and so on.

Why aliases?

```
FROM student AS s CROSS JOIN department AS d
```

- `s.name` → clearly from `student`
- `d.department_name` → clearly from `department`
- If both had a column `name`, without aliases MySQL would not know which you mean

`AS` is optional in MySQL (`student s` also works); writing `AS` is clearer for readers.

Purpose (class)

Generate **all possible combinations** — e.g. "every student paired with every department" (probability / option space of where someone *could* get admission). Rare in day-to-day reporting; crucial to understand because other joins are "Cartesian product then filter."

INNER JOIN and Equi Join

INNER JOIN — only matching rows

Returns rows where the **join condition finds a match in both tables**.

```
SELECT s.student_id, s.name, d.department_name
FROM student AS s
INNER JOIN department AS d
    ON s.department_id = d.department_id
ORDER BY s.student_id;
```

Student	department_id	Appears?
Rahul (10)	Computer	Yes
Priya (20)	IT	Yes
Sneha (NULL)	—	**No** — nothing equals NULL in the ON condition

Business meaning: "List students **who have** a department, with the department name." Students with no admission vanish.

Why not read department_id alone?

From `student` you see `10` but not the word "Computer". The join **looks up** the name in `department`. That is the everyday reason for INNER JOIN.

Equi join

An **equi join** is a join whose matching condition uses **only equality (`=`)** between columns.

In practice, the everyday `INNER JOIN … ON a.id = b.id` **is** an equi join. Syntax often written as plain `JOIN` (INNER is default):

```
SELECT s.student_id, s.name, d.department_name
FROM student AS s
JOIN department AS d
    ON s.department_id = d.department_id;
```

Same result as INNER JOIN for this equality condition.

	INNER JOIN	Equi join
Condition operators	Any (`=` , `>` , `<` , `BETWEEN` , …)	**Only `=`**
Class example	Equality on `department_id`	Same result here

When you use `>` instead of `=` , you have an INNER JOIN that is a **non-equi** join (next sections) — still "inner" (only matches), but not "equi."

Old Join Syntax (WHERE) and USING

Comma + WHERE (older style)

Before the `JOIN` keyword was common, people wrote:

```
SELECT s.student_id, s.name, d.department_name
FROM student AS s, department AS d
WHERE s.department_id = d.department_id;
```

Same result as INNER JOIN for this case. Class called it the **older way** (you used it with passport tables on Day 3). Prefer `JOIN … ON` in new code — clearer that you mean a join, not a filter-only WHERE.

USING — same column name on both sides

When the join column has the **exact same name** in both tables:

```sql
SELECT s.student_id, s.name, d.department_name
FROM student AS s
JOIN department AS d
USING (department_id);
```

Equivalent to `ON s.department_id = d.department_id`.

Caveat: if *multiple* columns share names, `USING` / NATURAL may match on **all** of them — only use when that is what you want.

NATURAL JOIN

NATURAL JOIN automatically joins on **all columns that have the same name** in both tables. You do **not** write `ON` or `USING`.

```sql
SELECT s.student_id, s.name, d.department_name
FROM student AS s
NATURAL JOIN department AS d;
```

Here both tables share `department_id` → join on that.

	USING (col)	NATURAL JOIN
You list columns	Yes	No — all same-named columns
Risk	Low if you pick one column	Higher if extra shared names appear later

For learning and production, `JOIN … ON` is usually safest and most explicit.

Non-Equi Join

Join condition uses **comparison other than equality** (or a range), e.g. `>`, `<`, `BETWEEN`.

```sql
SELECT s.name,
       s.department_id AS my_department_id,
       d.department_id AS other_department_id,
       d.department_name
FROM student AS s
JOIN department AS d
    ON s.department_id > d.department_id;
```

Example reading:

- Rahul has `department_id = 10` → no department id **less than** 10 → Rahul may not appear
- Priya has `20` → matches department `10` (Computer) once
- Karan has `30` → matches `10` and `20` → **two** result rows

Purpose: match using a **range / inequality**, not "same key." Real use cases include salary bands, date ranges (`BETWEEN`), rating scales — the class demo on `department_id` is for understanding, not a business rule.

You can also put non-equi conditions on LEFT/RIGHT joins when the application needs it — the result shape changes accordingly.

LEFT JOIN and RIGHT JOIN

LEFT JOIN (LEFT OUTER JOIN)

Return **every row from the left** table, and matching rows from the right. If no match → right-side columns are **NULL**.

```
SELECT s.student_id, s.name, d.department_name
FROM student AS s
LEFT JOIN department AS d
    ON s.department_id = d.department_id
ORDER BY s.student_id;
```

Student	Effect
Rahul (10)	Computer
Sneha (NULL)	Still listed; `department_name` is **NULL**

Purpose: keep every student in the output **even if** they have no department.

In INNER JOIN, Sneha disappeared. In LEFT JOIN, she stays.

RIGHT JOIN (RIGHT OUTER JOIN)

Return **every row from the right** table, and matching rows from the left. If no match → left-side columns are **NULL**.

```
SELECT s.student_id, s.name, d.department_name
FROM student AS s
RIGHT JOIN department AS d
    ON s.department_id = d.department_id
ORDER BY s.student_id;
```

Department	Effect
Computer / IT / Electronics	Appear with their students
Mechanical (40)	Appears with `student_id` / `name` = **NULL** (nobody admitted)
Sneha	Does **not** appear — she is only on the left, with no department match on the right

Mental model

```
LEFT JOIN:   keep all LEFT  + matches from right (else NULL on right)
RIGHT JOIN:  keep all RIGHT + matches from left  (else NULL on left)
INNER JOIN:  keep only matches
```

Most people prefer writing **LEFT JOIN** and swapping table order instead of RIGHT JOIN — same power, one habit.

FULL OUTER JOIN in MySQL

Full outer join = all left rows + all right rows: matches where possible, NULLs where not.

MySQL does not support `FULL OUTER JOIN` directly. Class pattern: **LEFT JOIN UNION RIGHT JOIN**.

```sql
SELECT s.student_id, s.name, d.department_name
FROM student AS s
LEFT JOIN department AS d
    ON s.department_id = d.department_id

UNION

SELECT s.student_id, s.name, d.department_name
FROM student AS s
RIGHT JOIN department AS d
    ON s.department_id = d.department_id;
```

Result includes:

- All students (with NULL department if needed) — from LEFT
- Mechanical with NULL student — from RIGHT
- UNION removes exact duplicate overlap of the "matched" rows

Use the **same selected columns** and **same ON condition** on both sides. Give aliases consistently (`student AS s`) on both branches so the parser stays happy.

SELF JOIN

A **self join** joins a table **to itself**. Needed when rows relate to other rows in the **same** table.

Class model: `mentor_id` references `student.student_id` — Priya's mentor is Rahul (`mentor_id = 1`), etc.

Aliases are mandatory — you must pretend there are two copies: `s` (student) and `m` (mentor).

```
SELECT
    s.name AS student_name,
    m.name AS mentor_name
FROM student AS s
LEFT JOIN student AS m
    ON s.mentor_id = m.student_id
ORDER BY s.student_id;
```

student_name	mentor_name
Rahul	NULL (no mentor)
Priya	Rahul
Amit	Rahul
Sneha	Priya

Purpose: compare / connect records **within** one table (org chart, mentor–mentee, reply-to-comment, manager–employee).

LEFT vs RIGHT self-join can change who appears with NULLs — practice both and read the result carefully.

Joining Three Tables

Many-to-many between students and courses is usually modeled with a **junction** table:

```
student ──< student_course >── course
   (1)         (many)            (1)
```

- `course` — parent of course catalog
- `student` — parent of people
- `student_course` — child holding `(student_id, course_id)` FKs

```sql
-- Sketch
CREATE TABLE course (
    course_id   INT PRIMARY KEY,
    course_name VARCHAR(30)
);

CREATE TABLE student_course (
    student_id INT,
    course_id  INT,
    FOREIGN KEY (student_id) REFERENCES student(student_id),
    FOREIGN KEY (course_id)  REFERENCES course(course_id)
);
```

Rahul (id 1) takes Java (101) and MySQL (102) → two rows in `student_course`.

Query: student name + course name

```sql
SELECT s.student_id, s.name, c.course_name
FROM student AS s
JOIN student_course AS sc
    ON s.student_id = sc.student_id
JOIN course AS c
    ON sc.course_id = c.course_id;
```

Chain of joins: student → bridge → course. As long as FKs connect the tables, you can join **many** tables (no fixed small maximum in practice).

Window Functions vs GROUP BY

The limitation of GROUP BY

```sql
SELECT department, SUM(salary) AS total
FROM employee
GROUP BY department;
```

You get **one row per department**. You **cannot** easily put each employee's salary **next to** the department average on the same row using only GROUP BY — grouping collapses detail rows.

What window functions fix

A **window function** calculates a value **across a set of rows** (the "window") **without collapsing** those rows. Each employee stays visible; extra columns show averages, ranks, running totals, etc.

```sql
CREATE TABLE employee (
    emp_id       INT PRIMARY KEY,
    emp_name     VARCHAR(30),
    department   VARCHAR(30),
    salary       INT,
    joining_date DATE
);
```

OVER, PARTITION BY, ORDER BY

OVER () — whole result as one window

```sql
SELECT emp_name, salary,
       AVG(salary) OVER () AS company_avg_salary
FROM employee;
```

Every row shows the **same** company-wide average next to the individual's salary. With GROUP BY alone you would lose the per-employee rows.

OVER (PARTITION BY ...) — like GROUP BY but keep rows

```sql
SELECT emp_name, department, salary,
       AVG(salary) OVER (PARTITION BY department) AS dept_avg_salary
FROM employee;
```

Each employee sees the average **of their department**. Compare: Amit earns 60,000 but dept average is 65,000 → below average.

`PARTITION BY` is the window cousin of `GROUP BY`.

OVER (ORDER BY ...)

Orders rows **inside** the window for ranking / running calculations:

```sql
SELECT emp_name, salary,
       AVG(salary) OVER (ORDER BY salary, emp_id) AS  ...
FROM employee;
```

When salaries tie, secondary sort on `emp_id` breaks ties consistently.

Alias naming warning (class)

Do **not** alias a window column with a reserved-looking name like `row_number` or `rank` if MySQL complains:

```sql
-- Risky
ROW_NUMBER() OVER ( ... ) AS row_number

-- Safer
ROW_NUMBER() OVER ( ... ) AS rn
RANK() OVER ( ... ) AS salary_rank
```

Running Totals and Moving Windows

Running total — from start through current row

```sql
SELECT emp_name, salary,
       SUM(salary) OVER (
           ORDER BY salary, emp_id
           ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS running_total
FROM employee;
```

Idea	Meaning
`UNBOUNDED PRECEDING`	Start at the **first** row of the window
`CURRENT ROW`	Through the row you are on

Example: 40k → running 40k; next 45k → running 85k; next 50k → 135k; …

Last N rows only — e.g. 2 preceding

```sql
SELECT emp_name, salary,
       SUM(salary) OVER (
           ORDER BY salary, emp_id
           ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
       ) AS moving_sum
FROM employee;
```

Only the **current row + up to 2 previous** rows are summed (not the entire history). Early rows have fewer predecessors.

Frame	Behaviour
`UNBOUNDED PRECEDING … CURRENT ROW`	Growing running total from start
`2 PRECEDING … CURRENT ROW`	Sliding window of width ≤ 3

Sales “running profit / cost” style reports use these patterns heavily.

ROW_NUMBER, RANK, DENSE_RANK, NTILE

ROW_NUMBER()

Gives a unique sequence number per row in the ordered window — **ties still get different numbers**.

```
SELECT emp_name, salary,
       ROW_NUMBER() OVER (ORDER BY salary, emp_id) AS rn
FROM employee;
```

Even if two people share a salary, they get 5 and 6, not the same number.

RANK()

Same salary → **same rank**; next rank **skips**.

```
SELECT emp_name, salary,
       RANK() OVER (ORDER BY salary DESC) AS salary_rank
FROM employee;
```

If Priya and Rahul both rank 1 → next person is rank **3** (2 is skipped).

Seniority by joining date:

```
SELECT emp_name, joining_date,
       RANK() OVER (ORDER BY joining_date) AS seniority_rank
FROM employee;

-- Senior-most only
SELECT emp_name, joining_date
FROM employee
ORDER BY joining_date
LIMIT 1;

-- Second senior-most
SELECT emp_name, joining_date
FROM employee
ORDER BY joining_date
LIMIT 1 OFFSET 1;
```

DENSE_RANK()

Same salary → same rank; next rank **does not skip**.

```sql
SELECT emp_name, salary,
       DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_salary_rank
FROM employee;
```

Two people at rank 1 → next is rank **2** (not 3).

Function	Ties	Next value
`ROW_NUMBER`	Different numbers	Always +1
`RANK`	Same number	Skips
`DENSE_RANK`	Same number	No skip

NTILE(n) — split into n buckets

```sql
SELECT emp_name, salary,
       NTILE(4) OVER (ORDER BY salary DESC, emp_id) AS salary_bucket
FROM employee;
```

Divides ordered rows into **4** roughly equal groups (elite / gold / silver style bands). `NTILE(2)` → two halves.

If 10 rows and `NTILE(4)` → buckets sized about 3,3,2,2. Ties can fall into adjacent buckets when the split point lands between equal salaries — use a tie-breaker in `ORDER BY` (`emp_id`) for stability.

Window Aggregates: SUM, AVG, COUNT

Same aggregates as Day 3, but with `OVER` :

```sql
-- Company total next to each employee
SELECT emp_name, salary,
       SUM(salary) OVER () AS company_total
FROM employee;

-- Department total next to each employee
SELECT emp_name, department, salary,
       SUM(salary) OVER (PARTITION BY department) AS dept_total
FROM employee;

-- Headcount per department on every row
SELECT emp_name, department,
       COUNT(*) OVER (PARTITION BY department) AS employees_in_dept
FROM employee;
```

Difference from GROUP BY: window keeps **one output row per input row**; GROUP BY returns **one row per group**.

LAG and LEAD

Function	Returns
LAG(expr)	Value from the **previous** row in the window order
LEAD(expr)	Value from the **next** row in the window order

```sql
SELECT emp_name, salary, joining_date,
       LAG(salary)  OVER (ORDER BY joining_date, emp_id) AS previous_salary,
       LEAD(salary) OVER (ORDER BY joining_date, emp_id) AS next_salary
FROM employee
ORDER BY joining_date, emp_id;
```

- First row → `previous_salary` is **NULL** (no previous)
- Last row → `next_salary` is **NULL** (no next)
- You can `LAG(joining_date)` / `LEAD(joining_date)` the same way — any expression, not only salary

Use cases: compare today vs yesterday, detect salary jumps, "previous order date," etc.

CTEs — Common Table Expressions

What is a CTE?

A **CTE** (`WITH … AS (…)`) names a temporary **result set** for the **next** SQL statement. Think: "a disposable named subquery / mini-table."

```sql
WITH overall_average AS (
    SELECT AVG(salary) AS avg_salary
    FROM employee
)
SELECT e.emp_name, e.department, e.salary
FROM employee AS e
CROSS JOIN overall_average AS oa
WHERE e.salary > oa.avg_salary;
```

- CTE holds company average once
- Main query lists who earns **above** that average

- After this statement finishes, the CTE is **gone**

CTE vs temporary table

	CTE	TEMPORARY TABLE
Lifetime	Current statement (then discarded)	Until `DROP` or session ends
Syntax	`WITH name AS (SELECT …)`	`CREATE TEMPORARY TABLE`
Reuse in same session later	Must rewrite `WITH`	Can `SELECT` again until dropped
Best for	Readable multi-step queries	Heavier intermediate materialization

Chained (multiple) CTEs

```sql
WITH department_average AS (
    SELECT department, AVG(salary) AS avg_salary
    FROM employee
    GROUP BY department
),
above_average AS (
    SELECT e.emp_name, e.department, e.salary, d.avg_salary
    FROM employee AS e
    JOIN department_average AS d
        ON e.department = d.department
    WHERE e.salary > d.avg_salary
)
SELECT *
FROM above_average
ORDER BY department, salary DESC;
```

First CTE builds per-department averages; second CTE finds people above their **department** average; final `SELECT` reads the second CTE. Only one `WITH` keyword — separate CTEs with commas.

Recursive CTE

A CTE that **refers to itself** — classic for sequences, trees, org charts.

```sql
WITH RECURSIVE numbers AS (
    SELECT 1 AS n                  -- anchor (base case)
    UNION ALL
    SELECT n + 1                   -- recursive member
    FROM numbers
    WHERE n < 10                   -- stop condition (critical!)
)
SELECT n FROM numbers;
-- 1,2,3,…,10
```

Part	Role
Anchor	Starting row(s)
`UNION ALL` + recursive SELECT	Builds next rows from previous
`WHERE` stop	Prevents infinite recursion

Without a stop condition, recursion would loop until the server hits a recursion limit / error.

Day Wrap and Practice Checklist

Relational algebra → SQL

Algebra	Symbol	SQL
Selection	σ	`WHERE`
Projection	π	`SELECT` column list
Union	∪	`UNION` / `UNION ALL`
Intersection	∩	`INTERSECT`
Difference	−	`EXCEPT`

Joins cheat-sheet

Join	Keeps
CROSS	All combinations (m × n)
INNER / equi	Matches only
LEFT	All left + matches (NULL on right)
RIGHT	All right + matches (NULL on left)
FULL (emulated)	LEFT UNION RIGHT
SELF	Same table twice (aliases)
NON-EQUI	Match with `>`, `<`, `BETWEEN`, …
NATURAL / USING	Match on same-named column(s)

Window & CTE

- `OVER()`, `PARTITION BY`, `ORDER BY`
- Running / moving sums with `ROWS BETWEEN …`
- `ROW_NUMBER` / `RANK` / `DENSE_RANK` / `NTILE`
- `LAG` / `LEAD`
- `WITH` CTE, chained CTEs, `WITH RECURSIVE`

Practice (strongly recommended)

1. Rebuild department/student data; run every join type; predict NULL rows before looking.
2. Write UNION / INTERSECT / EXCEPT on two projections; swap EXCEPT order.
3. Self-join mentors; 3-table student–course query.
4. For `employee`: dept avg via window; rank by salary; running total; lag salary by joining_date.
5. CTE: earners below company average; chained CTE for above **department** average.
6. Recursive CTE: print 1..N; then try to sketch an org-tree style recursion.

> Joins + windows are **interview favourites**. Pace felt fast in class — slow practice with your own screenshots of result grids is the way these stick.
> Coverage: DBT Day 4 lecture (12 Oct 2026) — expanded from the Hinglish transcript into step-by-step English notes with runnable MySQL patterns.